FK-only Leg Odometry

一句话

用 IMU 的陀螺仪给姿态，用腿的正向运动学（FK）给前进速度，两者相乘就得到机器人在世界里每秒走多远。没有 Kalman 滤波，一共 9 个 state 标量。

在三条实测 bag 上的结果：走路场景 XY 距离误差 < 1%，形状 RMSE < 2 m / 63 m 路径，Z 漂 < 5 cm。

核心思路

1. 腿告诉我们"身体怎么相对脚运动"

URDF 的正向运动学（FK）给出：**脚在身体坐标系里的位置** `fk_body = FK(q)`，`q` 就是 12 个腿关节的编码器读数。

对这个做时间导数，得到 **脚在身体系里的线速度**：

```
v_foot_body = J(q) · q̇
```

`J` 是 Jacobian，`q̇` 是关节角速度（编码器直接给）。KDL 库直接算。

2. 当脚贴地时，这告诉我们身体在动

Stance 时脚在世界里是**固定**的，所以：

```
脚_world = 身体_world + R · 脚_身体   →   恒定
对时间求导:  0 = v_身体_world + R · v_foot_body
```

所以：

```
v_身体_world = -R · v_foot_body
```

一只脚贴地就够算，两脚都贴地时取平均。两脚都离地（swing）时没有信息，就让 `v_世界 = 0`。

3. 身体姿态 R 怎么来

`R`（身体到世界的旋转）**不能从 FK 算**（下一节细说），只能靠 IMU：

- **Pitch / Roll**：陀螺仪积分（每帧 `R ← R · exp(ω·dt)`） + 加速度计在准静态时把它拉回 gravity 方向（Mahony 补偿）
- **Yaw**：纯陀螺仪积分，**长时间必漂**，需要外部传感器（LiDAR、磁力计）才能绝对锁定

4. 位置

```
p ← p + v_世界 · dt
```

就一行积分。加一个 FlatZ 约束：走路时一直把 `p.z` 低通拉回 0（办公室地板平坦先验）。

为什么 R 只能 gyro 积分，不能用 FK 更新

直觉理由

FK 告诉我们的是 **"给定 R 的话，身体在世界里速度是多少"**，不是 **"R 本身是多少"**。

```
v_身体_世界 = -R · v_foot_body
```

这条方程里 `R` 是**乘数**，不是**未知数**。不给 `R` 一个外部参考，方程里的 R 无法被反解出来。

是不是能用"连续几步的脚落点"反推 R？

直觉：脚每步落在地上某个位置，把这些位置连起来就知道机器人朝哪走。但：

- 我们**没有外部手段**测量"脚落在世界哪里"（那正是我们要求的）
- 脚的世界位置 = `p_身体 + R · 脚_身体`，两个未知量（p 和 R）都要估
- 用估出来的 p 反推 R → **循环依赖**，估计漂的时候脚落点跟着漂，反推出的 R 也是错的

之前的 EKF 版本试过这条路径，结果**越纠越歪**，删除了。

那 pitch / roll 为什么可以用 accel 修？

因为 gravity 是外部绝对参考。地球重力方向永远指向地心，加速度计在身体静止时就在测这个方向。比较：

- "gravity 在 body 里的方向" ← accel 读数可以告诉我们
- "gravity 在 world 里的方向" ← 定义就是 (0, 0, −9.81)

两者一比，就能推出 R 的 pitch/roll 部分。这不是循环，**是外部绝对方向**。

但 **yaw 这件事 accel 管不到** —— 绕 z 轴转一圈，gravity 在 body 里的读数一点不变。同理陀螺仪 / FK / contact 都管不到 yaw 绝对值，**只能靠磁力计、LiDAR/视觉 SLAM、GPS 这类真正有外部 heading 参考的传感器**。

能用 FK 做 R 更新吗？

唯一不需要外部 heading 就能约束 R 的 FK 情形是：**如果我们知道 "脚在世界里是否完全是平的"**（完全贴地，不是仅 ankle 一点贴地），那完全贴地的 world orientation = ground plane orientation（= 已知水平），这就约束了 R 的 pitch/roll。

但：

- 我们的 contact detection 只知道"脚是否接触"，不知道 "是否完全平贴"（heel-strike 时只有脚跟接触，sole 是斜的）
- 真的要做，就要一套 "sole-normal-from-accel-under-stance" 的额外估计，比 Mahony 加 accel 复杂几倍，没有必要

所以：**R 的 yaw 只能靠 gyro 积分，时间长了肯定会有问题；pitch/roll 可以靠 accel 拉**，就这样。这是本方案的**物理硬边界**，不是工程取舍。

三个信号流

这个方案算不上"滤波"，更像三条并行管道各自走：

```
IMU gyro ———→ R (gyro 积分) ←—— accel(Mahony 拉 tilt)
                 |
                 ▼
joint q,q̇ —→ FK: v_foot_body = J(q)·q̇
                 |
                 ▼
joint effort —→ ContactDetector → cl, cr (布尔)
                 |
                 ▼
           v_世界 = -R · v_foot_body  (按 cl, cr 选脚)
                 |
                 ▼
           p ← p + v · dt  (+ FlatZ 钳制 z)
```

核心代码就是 `joint_cb`，每收到一条 `/joint_states` 消息跑一次。

实测数据

三条 bag，都是实机录的：

bag	场景	时长	GLIM 参考路径	leg_odom 路径	path ratio	Kabsch 对齐 RMSE	Z 漂
16_12_13	103 办公室，走环	184 s	62.73 m	**63.02 m**	**1.005**	1.42 m	3.5 cm
17_15_31	B1 停车场，走环	180 s	103.67 m	**104.30 m**	**1.006**	5.35 m	4.5 cm
04_10-18_56_38	原地转圈	78 s	11.68 m	13.21 m	1.131	0.10 m	3.4 cm

（前两条 bag 用 `foot_roll_toe_offset=0.20`，转圈 bag 用 `0.0`，参数意义见下节）

走环场景 path ratio ~1.006（距离偏差 < 1 %）**是跨 bag 一致的**，说明算法稳定。

转圈场景 path ratio 1.131（多报 13 %）—— 这是已知局限，因为脚板 pivot 的效应 FK 看不到，当前一个 linear 补偿模型不能同时覆盖"走路"和"转圈"两种情形。要 fix 需要更精细的"真在前进吗"判据，暂不做。

bag 3 RMSE 5.35 m 比 bag 2 大 3 倍，原因是 bag 3 的陀螺仪 bias 估计偏大（−0.00852 rad/s vs bag 2 的 0.00005），180 s 累计 88° yaw 漂，把后半段形状拖歪。不是算法 bug，是 gyro bias 温漂的物理极限。想治这个只能外部闭环（GLIM yaw / 磁力计 / 闭环 SLAM）。

使用

装 package 后直接 launch

```
# 向前走（办公室、停车场、直线）
ros2 launch leg_odometry fk_only_node.launch.py \
    urdf_path:=/abs/path/to/casbot02_7dof_shell.urdf \
    foot_roll_toe_offset:=0.20

# 原地转圈（或场景不确定，保守值）
ros2 launch leg_odometry fk_only_node.launch.py \
    urdf_path:=/abs/path/to/casbot02_7dof_shell.urdf \
    foot_roll_toe_offset:=0.0
```

订阅 / 发布

- 订阅：`/imu`（`sensor_msgs/Imu`）、`/joint_states`（`sensor_msgs/JointState`）
- 发布：`/leg_odometry`（`nav_msgs/Odometry`）、TF `odom → base_link_leg_odom`

关键参数

参数	默认	作用
`bias_window_sec`	3.0	起动静止段长度（估 gyro bias + 初始化 R），要求开机前 3 s 机器人静止
`effort_threshold_{left,right}`	5.0, 5.0	接触检测阈值（Nm），左右不对称时可各自调
`tilt_kp`	1.0	Mahony 补偿增益
`tilt_accel_band`	0.5	accel 准静态门控，||a|-9.81| < 这个值才触发 tilt 修正
`flatz_enabled`	true	FlatZ 钳制开关
`flatz_alpha`	0.05	每帧 p.z 向 0 收敛的比例（α=0.05 时时间常数 ~0.1 s）
`foot_roll_toe_offset`	0.0	**关键**：heel-toe rolling 补偿，走路 0.20，转圈 0.0

场景 → 参数对照

场景	foot_roll_toe_offset	flatz_alpha
室内平地走路	0.20	0.05
室内原地转圈	0.0	0.05
室外不平地面走	0.10-0.20（调）	0.02（或 0 关掉）
上下坡 / 楼梯	不建议用 FlatZ（`flatz_enabled:=false`）	—

已知局限

1. **Yaw 长时间必漂** —— 这是物理硬边界，没有外部 heading 传感器就是没法治。本 bag 量级是 180 s ~88° 最坏
2. **原地转圈 path 过估 13 %** —— foot_roll 补偿对转圈场景是过补偿；手动设 0 规避
3. **假设开机前 3 s 静止** —— 不静止的话 gyro bias 和 gravity 初始化都有偏，建议启动流程固定 3 s 静止
4. **不处理上下坡** —— FlatZ 假设地面平。真上坡要把 flatz_enabled 关掉（会换来 z 方向的 IMU 漂）
5. **无下游 covariance** —— `/leg_odometry` 的 twist.covariance 和 pose.covariance 留空。要接严格的 SLAM 融合时需要补

代码结构

```
src/leg_odometry/
├── src/fk_only_node.cpp                    ← 核心节点，~380 行
├── include/leg_odometry/
│   ├── so3_utils.h                         skew, exp_so3
│   └── state/Kinematics.h                  LegKinematics, ContactDetector
├── launch/fk_only_node.launch.py
└── config/fk_only_params.yaml
```

joint_cb（每 5 ms 跑一次）的 8 步流程，对应上面的"核心思路"：

```
1. 更新 q, q̇, effort 缓存
2. 计算 dt
3. R 积分                    R_ = R_ * exp_so3((gyro - bg_) * dt)
4. FK 算脚速度               v_foot_body = kin_.foot_velocity_{left,right}
(q, q̇)
5. foot_roll 补偿            v_foot_body.x += toe_offset * q̇_ankle_pitch
6. 接触检测                  cl, cr = det_->update(eff_L, eff_R)
7. Mahony tilt 修 R          (门控: accel 准静态 + 至少单脚 stance)
8. 积分位置                  p += -R * v_foot_body * dt,  FlatZ 钳 p.z
```

Python 参考实现在 `scripts/fk_only_odometry.py`（离线读 bag），算法一致，做回归测试基准。

一句话总结

IMU 给姿态、腿给速度、方法简单透明、已知上限是 yaw 长漂。够用则用，要治 yaw 就接外部。